UiT Norges arktiske universitet

SENSORVEILEDNING

Kurskode:	INF-0103
Eksamensdato:	2025/12/11
Fagperson/ intern sensor:	Einar Holsbø
Telefon/e-mail:	48032339 einar.j.holsbo@uit.no
Antall sider: inkl. forside	5

Eksamen i INF-0103, Høst 2025.

Les oppgaveteksten grundig og disponer tiden slik at du får tid til å svare på alle oppgavene.

I noen oppgaver kan det være nødvendig å tolke oppgaveteksten ved å gjøre noen antagelser. Gjør i så fall rede for hvilke antagelser du har gjort, men pass på å ikke gjøre antagelser som trivialiserer oppgaven.

Der du skal utvikle eller beskrive en algoritme anbefales det at du først beskriver algoritmen på et høyt abstraksjonsnivå, f.eks. med ord og figurer, før du går videre med detaljer og eventuell pseudokode.

Der du blir spurt om å implementere noe kan du skrive C-lignende pseudokode. Skrivefeil eller syntaxfeil trekker ikke ned.

Husk at du kan referere tilbake til funksjoner du tidligere har definert.

Oppgave 1 ($33 + \frac{1}{3}\%$)

Gi en kort beskrivelse av von Neumann modellen og instruksjonssyklusen. Beskrivelsen bør omfatte de ulike komponentene i modellen og hvordan disse interagerer med hverandre.

Sensorveiledning

Alle fem komponenter skal være kort beskrevet, og helst supplementert med en tegning/figur. Det er flere måter å tegne von Neumann-modellen, og vi har vært tydelige på i forelesning at tegningen i boka til Patt & Patel ikke er den eneste riktige tegningen.

Det er ikke så nøye f.eks. nøyaktig hvor registre er plassert så lenge det gir mening. For eksempel kan MAR/MDR like gjerne være et sted i CPUen (eller kontroll- eller prosesseringsenhetene om de tegnes separat), som i minnet. Bussene bør være tegnet inn med piler mellom komponentene.

Komponenter å få med seg:

- Input/output med eksempler på hva disse enhetene kan være.
- Kontrollenhet, med sine to registre; PC og IR/CIR, hva disse brukes til, og hvordan disse brukes. Gjerne beskrevet i instruksjonssyklusen.
- Prosesseringsenheten med ALU og registre (TEMP/ACC). Hvordan registre brukes ifm. ALUens utføring av instruksjoner.
- Minnet og registrene som hører til her: MAR og MDR. Interaksjon mellom CPU og minne, hvordan man leser/skriver.

Kandidaten bør ha beskrevet at både data og instruksjoner er lagret i det samme minnet, og overføres over den samme databussen.

Stegene i instruksjonssyklusen skal være forklart på et detaljenivå hvor de nevner hvordan flere av de sentrale registrene brukes underveis. Vi godtar både den

"lange" instruksjonnsyklusen (fetch, decode, evaluate adddress, fetch operand, execute, store result) og den "korte" (fetch, decode, execute).

Detaljnivået i den "korte" syklusen bør omtrent tilsvare den "lange" beskrivelsen, slik at studentene beskriver oppføreselen til fetch operand og store result implisitt (i de tilfellene det er relevant) som en del av execute, uten at vi nødvendigvis krever at de bruker nøyaktig disse engelske begrepene.

Evaluate address har vi ikke fokusert særlig på, og det trekkes ikke for å sløyfe dette steget i besvarelsen. Vi abstraherer det inn i decode-fasen. I studentenes beskrivelse er det en fordel om de nevner bussene i kommunikasjon mellom komponentene. Klokkesykluser er ikke nødvendig å nevne.

Oppgave 2 $(33 + \frac{1}{3}\%)$

Gitt følgende funksjon

```
char *ukjent(char *c) {
    char *d = malloc(sizeof(char) * 100);
    char *r = d;
    char *e = c;
    int i = 0;

    while(*e) {
        i++;
        e++;
    }

    e--;

    while(i > 0) {
        *d = *e;
        d++;
        i--;
        e--;
    }

    return r;
}
```

a) Beskriv det funksjonen `ukjent` returnerer hvis vi kaller den slik:

```
ukjent("Hello world!");
```

b) Hva er poenget med `e--;` mellom `while`-løkkene?

c) Hvorfor returneres `r` og ikke `d`?

d) Hva skjer hvis vi gjør `ukjent(str);` hvor `str` er en streng som inneholder 128 karakterer?

Sensorveiledning

a) Funksjonen returnerer en peker til et `char` array som inneholder `"!dltow olleH"` i de første posisjonene. Man er ikke garantert noen nullterminering, så det er ikke teknisk sett en streng. Et perfekt svar er nøyaktig men studenten skal ha god uttelling for å skjønne at det er snakk om å reversere teksten.

b) Hvis man ikke dekrementerer `e` vil den terminerende `0` karakteren til strengen kopieres som det første i resultatarrayet.

c) Ved returtidspunkt peker ikke `d` til starten av arrayet lenger. Dette er hele poenget med å ha `r`, slik at man kan returnere en peker til starten av arrayet og ikke midt i.

d) Man skriver utenfor `d` og risikerer udefinert oppførsel.

Oppgave 3 ($33 + \frac{1}{3}\%$)

Et *dynamisk array* er en datastruktur som har plass til vilkårlig mange elementer. Dette får man til ved å allokere ekstra minne når det opprinnelige araryet begynner å bli fullt. Vi skal se på et dynamisk array med følgende interface:

```c
typedef struct arr arr_t;

// lager et nytt dynamisk array med plass til initial_size heltall
arr_t *arr_create(int initial_size);

// setter inn et tall my_number på plassen indikert av index i array
void arr_set(arr_t *array, unsigned int index, int my_number);

// henter ut det som ligger på plassen index i array
int arr_get(arr_t *array, unsigned int index);
```

Merk at arrayet skal inneholde heltall. I oppgavene nedenfor er du nødt til ta noen avgjørelser og gjøre noen antakelser. Beskriv disse.

a) Implementer datastrukturen `struct arr` som skal holde vårt dynamiske array.

b) Implementer funksjonen `arr_create`.

c) Implementer funksjonen `arr_get`.

d) Implementer funksjonen `arr_set`. Hint: Husk å tenke på hva som kan gå galt med `index` her.

Sensorveiledning

Deloppgave **d)** bør telle for omtrent halve oppgaven. De store linjene er det viktige her; halvveis utrygg kode er helt OK og småting som å ikke sjekke at allokeringer går bra eller off-by-one errors bør ikke telle for noe særlig.

a) Bør ha plass til noe data og bør ha styr på hvor mye data det er plass til

```
struct arr {
    int *data;
    unsigned int current_size;

};
```

b) For full uttelling bør man allokere plass til en ny `arr_t`, initialisere feltene i `struct arr` på fornuftig vis, og returnerer en peker.

```
arr_t *arr_create(int initial_size) {
    arr_t *new = malloc(sizeof(arr_t));
    new->current_size = initial_size;
    new->data = malloc(initial_size*sizeof(int));

    return new;
}
```

c) En lookup i `array->data`. Bør ha en sjekk for om `index` er utenfor minnet vi har allokert til `data` for full uttelling. Ikke så farlig med off-by-one errors og feil av den størrelsen.

```
int arr_get(arr_t *array, unsigned int index) {
    // merk: impliserer at 0 ikke er en gyldig størrelse i arrayet
    // hovedpoenget er å få med seg at det går an å havne utenfor
    if (index >= array->current_size) return 0;
    return array->data[index];
}
```

d) Tanken er at hvis `index` er større enn (eller lik) `current_size` må vi allokere mer plass. Man bør få med seg at nytt minne må allokeres, det gamle fris opp.

```
void arr_set(arr_t *array, unsigned int index, int my_number) {
    if (index >= array->current_size) {
        unsigned int newsize = index*3/2;
        int *newdata = malloc(newsize*sizeof(int));
        memcpy(newdata, array->data, array->current_size*sizeof(int));

        // her er rekkefølgen vesentlig, trekk for å sette array->data før
        // free(array->data)
        free(array->data);
        array->data = newdata;
        array->current_size = newsize;
    }

    array->data[index] = my_number;
}
```